

Titolo: Forza 4

Sottotitolo: Gioco in Java con interfaccia grafica e avversario controllato dal computer

Cognome Nome: Razvon Lukian

Data: 05/10/2026

Classe: 5D

Obiettivo: L'intento iniziale di questo lavoro era realizzare in Java una versione semplice di Forza 4 giocabile da riga di comando, con una griglia fissa e un solo avversario umano, principalmente per esercitarmi sulla logica di gioco e sul controllo di vittoria. Avendo già maturato una certa esperienza con la programmazione a oggetti, ho deciso di non fermarmi a questa versione minimale e di trasformare il progetto in un'applicazione grafica completa. L'obiettivo si è quindi evoluto verso la realizzazione di un vero e proprio gioco da tavolo digitale, con una griglia dalle dimensioni configurabili, tre modalità di gioco diverse (Umano contro Umano, Umano contro Computer, Umano contro un'intelligenza artificiale collegata via Internet) e un'interfaccia curata nei dettagli, con animazioni fluide, effetti sonori ed evidenziazioni visive.

Invece di scrivere tutta la logica in un'unica classe, ho strutturato il programma separando nettamente i ruoli: una classe che rappresenta lo stato della griglia (Griglia), una che si occupa solo del disegno e delle animazioni (GrigliaPanel), una che coordina lo svolgimento della partita (Partita) e un'interfaccia comune (Giocatore) che permette di trattare allo stesso modo un giocatore umano, un giocatore controllato dall'algoritmo Minimax e un giocatore controllato da un servizio di intelligenza artificiale esterno. Questa scelta architetturale ha permesso di aggiungere nuove modalità di gioco senza dover toccare la logica già scritta, sfruttando il polimorfismo.

Poiché le mosse dei giocatori umani dipendono dal clic del mouse, mentre l'interfaccia grafica deve restare sempre reattiva, ho dovuto affrontare anche il problema della concorrenza: la partita gira su un thread separato da quello dell'interfaccia grafica, e i due thread comunicano tra loro tramite un meccanismo di attesa e notifica.

Abstract: Il programma, scritto in Java con la libreria Swing, permette di giocare a Forza 4 su una griglia da 4 a 8 righe e colonne, in tre modalità: contro un'altra persona, contro il Computer (algoritmo Minimax con potatura alpha-beta) oppure contro un servizio di intelligenza artificiale esterno. Nella relazione descrivo come ho diviso il programma in

classi, i concetti teorici utilizzati, i problemi incontrati e come li ho risolti.

Corpo della relazione

Descrizione dell'approccio: Per sviluppare questo progetto ho seguito un approccio incrementale in quattro fasi. Nella prima fase ho definito con precisione il problema da risolvere: rappresentare una griglia di gioco generica (non necessariamente 7x6 come nel Forza 4 classico), gestire l'inserimento dei gettoni con l'effetto gravità, riconoscere le quattro direzioni di vittoria (orizzontale, verticale e le due diagonali) ed evitare che un'interfaccia grafica reattiva venisse bloccata dall'attesa delle mosse umane o dal calcolo dell'algoritmo Minimax.

Nella seconda fase ho progettato l'architettura del programma dividendo le responsabilità tra classi distinte. La classe Griglia contiene solo la logica di dati: una matrice `int[][]` che rappresenta lo stato delle celle (0 per una cella vuota, 1 o 2 per i gettoni dei due giocatori), i metodi per inserire un gettone rispettando la gravità, per verificare se una colonna è ancora giocabile, per controllare se la griglia è piena e per verificare la vittoria in tutte le direzioni. Ho scelto di non far dipendere questa classe da Swing o da qualsiasi elemento grafico, in modo che possa essere riusata anche, ad esempio, dall'algoritmo Minimax per simulare mosse future senza disegnare nulla.

La classe GrigliaPanel, che estende JPanel, si occupa invece esclusivamente della parte visiva: disegna il foglio a righe con il margine rosso, il tabellone tracciato a penna, i gettoni colorati a tratteggio come con un pastello, la colonna evidenziata sotto il mouse, l'animazione di caduta e, a fine partita, il banner col risultato e la cerchiatura della combinazione vincente. Questa separazione tra modello e vista ha reso più semplice, in un secondo momento, aggiungere nuove funzionalità puramente estetiche senza dover toccare la logica di gioco.

Per rendere intercambiabili i diversi tipi di giocatore ho definito l'interfaccia Giocatore, con due soli metodi: `sceglieMossa`, che restituisce la colonna scelta, e `getId`, che restituisce il numero del giocatore. Le classi `GiocatoreUmanoGUI`, `GiocatoreComputer` e `GiocatoreAI` implementano questa interfaccia in modi completamente diversi (rispettivamente aspettando un clic, interrogando l'algoritmo Minimax, oppure chiamando un servizio esterno), ma la classe Partita non ha bisogno di sapere quale dei tre sta usando: si limita a chiamare `sceglieMossa` sul giocatore di turno. Questo è un classico esempio di polimorfismo. È rimasta anche una vecchia classe, `GiocatoreUmano`, che leggeva le mosse da tastiera con uno Scanner: risale alla primissima versione a riga di comando del

progetto e non viene più utilizzata dall'interfaccia grafica, ma l'ho lasciata come testimonianza dell'evoluzione del lavoro.

Modello grafico: prima di procedere con la realizzazione dell'interfaccia grafica, abbiamo creato un modello grafico al computer, disegnando con un editor le due finestre del programma: la schermata iniziale con i campi da compilare e il pulsante per iniziare, e la finestra di gioco con il tabellone, la barra del turno in alto, il pulsante per tornare al menu e l'elenco delle mosse sulla destra. Progettare prima l'aspetto delle finestre ci ha permesso di decidere la disposizione dei componenti, le loro proporzioni e il comportamento atteso (ad esempio quali campi devono comparire in base alla modalità scelta) senza scrivere ancora una riga di codice, e di modificare facilmente le idee meno riuscite. Il modello è diventato poi il riferimento da seguire durante la scrittura di SchermataIniziale e MainGUI. Per la versione finale abbiamo scelto uno stile grafico che imita un disegno fatto a mano su un foglio a righe, con linee leggermente tremolanti, cerchi non perfetti e colori stesi a tratteggio.

Nella terza fase ho scritto effettivamente il codice, partendo dalla logica pura della griglia, poi l'interfaccia grafica, poi l'algoritmo Minimax e infine l'integrazione con il servizio di intelligenza artificiale esterno. Nella quarta fase ho verificato e corretto il programma giocando ripetutamente partite di prova, con griglie di dimensioni diverse, individuando via via i problemi descritti più avanti.

Cenni teorici:

Classe e oggetto: una classe è il modello che definisce quali dati e quali comportamenti avranno gli oggetti creati a partire da essa. Nel progetto, Griglia è la classe che descrive come è fatto un tabellone di gioco; ogni volta che si avvia una nuova partita, con `new Griglia(righe, colonne)`, viene creato un oggetto concreto che rappresenta quella specifica partita.

Interfaccia e polimorfismo: un'interfaccia definisce un contratto di metodi che le classi che la implementano devono rispettare, senza specificare come. L'interfaccia Giocatore permette alla classe Partita di gestire in modo identico un umano, il Computer e l'AI, chiamando sempre lo stesso metodo `sceglieMossa` e lasciando che sia l'oggetto concreto, a runtime, a decidere come comportarsi.

Incapsulamento: quasi tutti gli attributi delle classi del progetto sono dichiarati private, ed è possibile modificarli solo attraverso i metodi pubblici della classe stessa. Ad esempio, la

matrice celle dentro Griglia non è mai accessibile direttamente dall'esterno: la si può leggere con `getCella` e modificare solo tramite `inserisciGettone`, che applica anche la regola della gravità.

Thread e concorrenza: un thread è un flusso di esecuzione indipendente all'interno dello stesso programma. In questo progetto la partita gira su un thread dedicato (avviato in `MainGUI` con `new Thread(partita::avvia).start()`), separato dal thread dell'interfaccia grafica, perché il metodo `sceglieMossa` di un giocatore umano deve poter restare in attesa di un clic senza bloccare la finestra.

Sincronizzazione con `wait` e `notify`: quando un giocatore umano deve muovere, il thread della partita entra nel metodo `synchronized` `sceglieMossa` di `GiocatoreUmanoGUI` e chiama `wait()`, che sospende il thread e libera temporaneamente il monitor dell'oggetto. Quando l'utente clicca su una colonna, `GrigliaPanel` avvisa `MainGUI`, che chiama il metodo, anch'esso `synchronized`, `riceviClick`: questo salva la colonna scelta e chiama `notify()`, che risveglia il thread della partita rimasto in attesa. L'uso della parola chiave `synchronized` su entrambi i metodi garantisce che i due thread non possano leggere o scrivere la stessa variabile nello stesso istante.

Event Dispatch Thread e `SwingUtilities.invokeLater`: in Swing tutte le operazioni che modificano l'interfaccia grafica devono essere eseguite su un unico thread speciale, l'Event Dispatch Thread (EDT), altrimenti si rischiano comportamenti imprevedibili o blocchi dell'interfaccia. Poiché gli eventi di gioco (una mossa giocata, la fine della partita) vengono generati dal thread separato della Partita, ogni volta che occorre aggiornare un componente grafico (il pannello del tabellone, l'etichetta del turno, l'elenco delle mosse) il codice lo racchiude in `SwingUtilities.invokeLater`, che pianifica quell'operazione sull'EDT in modo sicuro.

Ricorsione e algoritmo Minimax: la ricorsione è la tecnica per cui una funzione richiama se stessa per risolvere un problema più piccolo dello stesso tipo. L'algoritmo Minimax simula in anticipo le possibili mosse future alternando un turno che massimizza il punteggio (quello dell'intelligenza artificiale) e un turno che lo minimizza (quello dell'avversario), esplorando l'albero delle mosse fino a una certa profondità e assegnando un punteggio euristico alle griglie raggiunte quando non conviene proseguire la ricorsione.

Potatura alpha-beta: è un'ottimizzazione dell'algoritmo Minimax che permette di interrompere l'esplorazione di un ramo dell'albero delle mosse non appena si dimostra che non potrà mai influenzare la decisione finale, riducendo drasticamente il numero di griglie da analizzare senza cambiare il risultato dell'algoritmo.

Timer di Swing: è un componente che permette di eseguire ripetutamente una porzione di codice a intervalli di tempo regolari, sul thread grafico. Viene usato per far avanzare, fotogramma dopo fotogramma, sia l'animazione di caduta dei gettoni sia l'animazione del banner e dell'evidenziazione a fine partita.

Coda (Queue): è una struttura dati che restituisce gli elementi nello stesso ordine in cui sono stati inseriti (FIFO, first in first out). Viene usata in GrigliaPanel per accodare le mosse che arrivano mentre un'animazione di caduta è già in corso, in modo che vengano poi animate una alla volta nell'ordine corretto, invece di sovrapporsi visivamente.

Gestione delle eccezioni (try-catch): è il meccanismo che permette di intercettare un errore che si verifica durante l'esecuzione senza far terminare bruscamente il programma. Viene usato, ad esempio, attorno alla chiamata di rete verso il servizio di intelligenza artificiale, in modo che un problema di connessione o una risposta non valida non blocchino la partita.

Richiesta HTTP e formato JSON: HTTP è il protocollo usato per comunicare con i servizi web; JSON è un formato testuale per scambiare dati strutturati. La classe ClienteAI costruisce una richiesta HTTP con la libreria `java.net.http`, inviando nel corpo un messaggio testuale in formato JSON che descrive la griglia attuale, e successivamente estrae dal JSON di risposta il testo generato dal servizio.

Descrizione dell'attività svolta: Il primo elemento realizzato è stata la classe Griglia, con la matrice di interi e i metodi per inserire un gettone (facendolo scendere fino alla prima riga libera, per simulare la gravità), per controllare se una colonna è valida e per verificare la vittoria. Il controllo di vittoria scorre la griglia cercando quattro gettoni consecutivi dello stesso giocatore in orizzontale, in verticale e nelle due diagonali; ho successivamente esteso questo controllo con il metodo `getLineaVincente`, che oltre a dire se qualcuno ha vinto restituisce anche le coordinate esatte dei quattro gettoni allineati, così da poterli evidenziare graficamente.

Ho poi realizzato l'interfaccia grafica con la schermata iniziale (`SchermataIniziale`), dove l'utente sceglie righe e colonne tramite due `JSpinner` limitati tra 4 e 8, la modalità di gioco tramite un menu a tendina, e, a seconda della modalità scelta, campi aggiuntivi che compaiono o scompaiono dinamicamente: i nomi dei due giocatori se si sceglie Umano contro Umano, oppure il campo per la chiave dell'API se si sceglie la modalità con l'intelligenza artificiale. Nella finestra di gioco vera e propria (`MainGUI`) ho aggiunto una barra superiore con il turno corrente e il pulsante per tornare al menu, e una barra laterale con la cronologia di tutte le mosse giocate, aggiornata in tempo reale.

FORZA 4



Righe 6



Colonne 7



Modalità Umano vs Umano



Nome Giocatore 1

Nome Giocatore 2

INIZIA

Figura 1: la schermata iniziale, con la scelta di righe, colonne, modalità e nomi dei giocatori.

Per l'animazione di caduta dei gettoni ho simulato una piccola fisica: la posizione verticale del gettone aumenta ad ogni fotogramma di una velocità che a sua volta cresce per

effetto di una costante di gravità, finché il gettone non raggiunge la riga finale; a quel punto rimbalza una volta, con una velocità ridotta rispetto a quella di arrivo, prima di fermarsi definitivamente. Poiché più mosse potrebbero arrivare mentre un'animazione è ancora in corso (ad esempio quando gioca il Computer subito dopo l'umano), le mosse in attesa vengono accodate in una Queue e animate una alla volta.

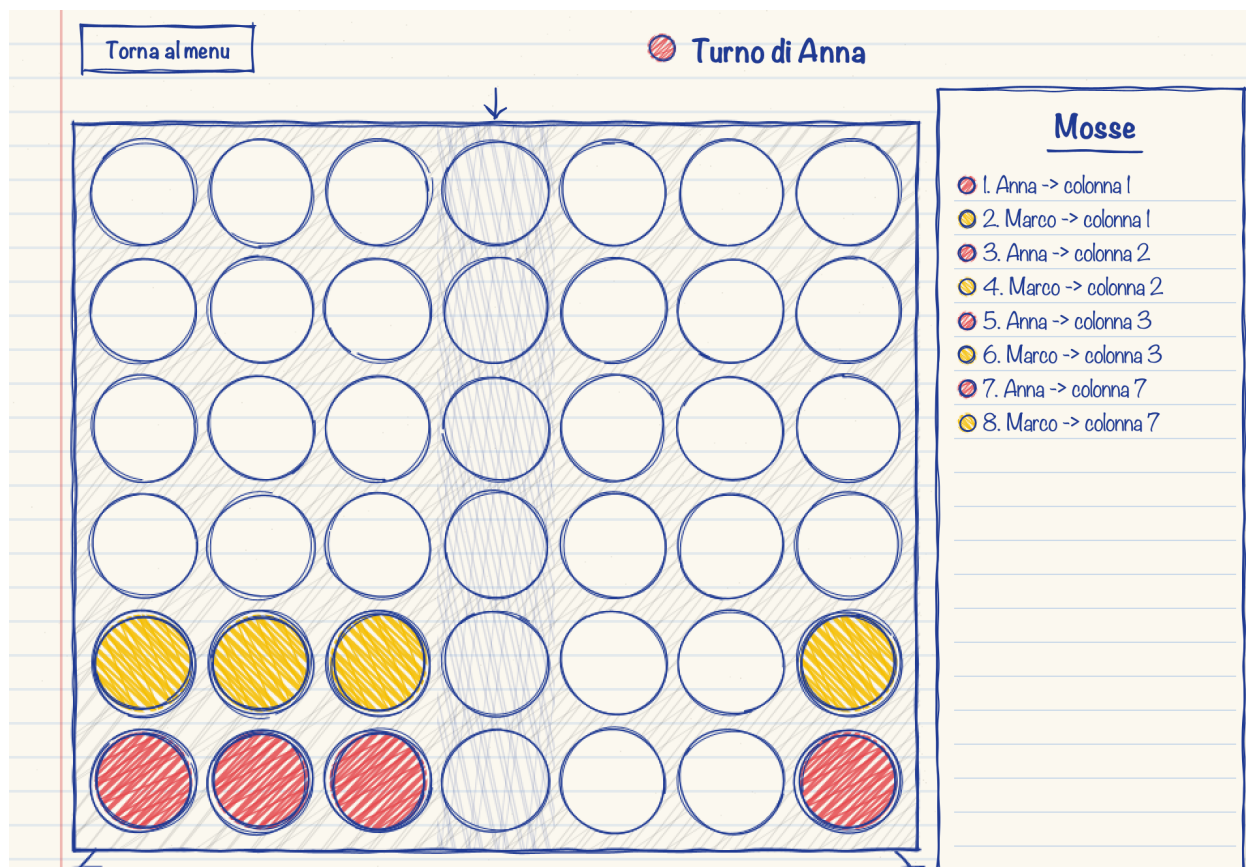


Figura 2: una partita in corso su una griglia 6x7, con la freccia sulla colonna sotto il mouse e la cronologia delle mosse sul foglietto laterale.

Per la modalità Umano contro Computer ho implementato l'algoritmo Minimax con potatura alpha-beta nella classe IA, con una profondità di ricerca fissata a 6 mosse. Per ogni colonna giocabile, l'algoritmo simula la mossa su una copia della griglia, ottenuta con `clonaGriglia`, in modo da non alterare mai la partita reale durante la simulazione, e prosegue ricorsivamente alternando i turni di massimizzazione e minimizzazione fino alla profondità stabilita o fino a fine partita. Quando la ricorsione si ferma, la griglia raggiunta viene valutata con una funzione euristica che assegna un punteggio a ogni possibile allineamento di quattro celle, premiando le configurazioni vantaggiose per l'intelligenza artificiale e penalizzando quelle vantaggiose per l'avversario.

```

16 public int scegliMossa(Gridlia griglia) {
17     int migliorColonna = -1; // colonna migliore trovata finora
18     int migliorValore = Integer.MIN_VALUE; // valore associato alla colonna migliore
19
20     for (int colonna = 0; colonna < griglia.getColonne(); colonna++) { // prova tutte le colonne
21         if (griglia.colonnaValida(colonna)) { // solo se la colonna non è piena
22
23             Gridlia copia = griglia.clonaGriglia(); // simula la mossa su una copia, senza toccare la partita vera
24             copia.inserisciGettone(colonna, idIA); // inserisce il gettone dell'IA nella copia
25
26             // profonditàMax - 1 perché questa mossa conta già come un livello
27             // false perché dopo la mossa dell'IA tocca all'avversario (turno di minimizzazione)
28             int valore = minimax(copia, profonditàMax - 1, false, Integer.MIN_VALUE, Integer.MAX_VALUE);
29
30             if (valore > migliorValore) { // se questa mossa è migliore di quella trovata finora
31                 migliorValore = valore; // aggiorna il valore migliore
32                 migliorColonna = colonna; // aggiorna la colonna migliore
33             }
34         }
35     }
36
37     return migliorColonna; // ritorna la colonna scelta dall'algoritmo
38 }
39
40 private int minimax(Gridlia griglia, int profondita, boolean turnoMax, int alpha, int beta) {
41     int risultato; // unico valore di ritorno della funzione, costruito nei tre rami
42
43     // condizioni di arresto della ricorsione: fine partita o profondità esaurita
44     if (profondita == 0 || griglia.verificaVittoria(idIA) || griglia.verificaVittoria(idAvversario) || griglia.grigliaPiena()) {
45         risultato = valutaGriglia(griglia); // valuta la griglia raggiunta
46
47         // a parità di esito conta QUANDO succede: una vittoria vicina vale più di una lontana,
48         // una sconfitta lontana vale più di una vicina. Senza questo, se l'IA vede di aver perso
49         // comunque smette di difendersi e gioca la prima colonna, perdendo subito
50         if (risultato == VINCITA) {
51             risultato += profondita; // più profondità rimasta = vittoria raggiunta prima
52         } else if (risultato == SCONFITTA) {
53             risultato -= profondita; // più profondità rimasta = sconfitta arrivata prima, quindi peggio
54         }
55     } else if (turnoMax) { // turno dell'IA: cerca la mossa che massimizza il punteggio
56
57         int migliore = Integer.MIN_VALUE; // parte dal valore più basso possibile
58         int colonna = 0;
59
60         // "beta > alpha" nella condizione del ciclo sostituisce il break: appena il ramo
61         // non può più migliorare il risultato, le colonne rimanenti non vengono più esplorate
62         while (colonna < griglia.getColonne() && beta > alpha) {
63             if (griglia.colonnaValida(colonna)) {
64
65                 Gridlia copia = griglia.clonaGriglia(); // simula la mossa
66                 copia.inserisciGettone(colonna, idIA); // gettone dell'IA
67
68                 int valore = minimax(copia, profondita - 1, false, alpha, beta); // esplora la mossa avversaria successiva
69                 migliore = Math.max(migliore, valore); // tiene il valore più alto trovato
70                 alpha = Math.max(alpha, migliore); // aggiorna alpha per il pruning
71             }
72             colonna++;
73         }
74
75         risultato = migliore;
76     } else { // turno dell'avversario: cerca la mossa che minimizza il punteggio dell'IA
77
78         int migliore = Integer.MAX_VALUE; // parte dal valore più alto possibile
79         int colonna = 0;
80
81         while (colonna < griglia.getColonne() && beta > alpha) {
82             if (griglia.colonnaValida(colonna)) {
83
84                 Gridlia copia = griglia.clonaGriglia(); // simula la mossa
85                 copia.inserisciGettone(colonna, idAvversario); // gettone dell'avversario
86
87                 int valore = minimax(copia, profondita - 1, true, alpha, beta); // esplora la mossa dell'IA successiva
88                 migliore = Math.min(migliore, valore); // tiene il valore più basso trovato
89                 beta = Math.min(beta, migliore); // aggiorna beta per il pruning
90             }
91             colonna++;
92         }
93
94         risultato = migliore;
95     }
96
97     return risultato;
98 }
99
100 }

```

Figura 3: in alto scegliMossa, il metodo chiamato quando si gioca contro il Computer, che avvia l'esplorazione su ogni colonna; in basso il metodo ricorsivo minimax con potatura alpha-beta che richiama se stesso, con le righe che premiano le vittorie più vicine e penalizzano le sconfitte più vicine, entrambi nella classe IA.

Per la modalità Umano contro AI ho realizzato la classe `ClienteAI`, che costruisce una richiesta HTTP verso un servizio di intelligenza artificiale esterno, descrivendo nel testo della richiesta lo stato attuale della griglia e chiedendo come risposta il numero della colonna da giocare. La classe `GiocatoreAI` si occupa di interpretare la risposta testuale, estraendo il primo numero presente, e di verificare che la colonna indicata sia davvero valida; se la richiesta fallisce per qualsiasi motivo (chiave mancante, problema di rete, risposta non interpretabile), il programma mostra un avviso una sola volta e prosegue comunque la partita scegliendo automaticamente la prima colonna libera, così una partita non può mai bloccarsi per un problema esterno.

Nella parte finale del lavoro ho curato l'aspetto e la piacevolezza dell'esperienza di gioco: ho aggiunto una coppia di effetti sonori generati direttamente dal programma (un breve "toc" quando il gettone tocca il fondo della colonna e un piccolo arpeggio a campanella quando qualcuno vince), sintetizzati calcolando matematicamente le onde sonore con la libreria `javax.sound.sampled`, senza quindi bisogno di alcun file audio esterno; ho sostituito la finestra di dialogo di fine partita con un banner animato disegnato direttamente sopra il tabellone, come un foglietto appoggiato sul quaderno, e ho aggiunto la cerchiatura a mano dei quattro gettoni della combinazione vincente. Infine ho ridisegnato tutta l'interfaccia in uno stile che imita il disegno a mano, raccogliendo le funzioni di disegno "a mano" (linee tremolanti, cerchi imperfetti, riempimenti a tratteggio) in una classe dedicata, `Matita`, usata sia dalla schermata iniziale sia dalla finestra di gioco.

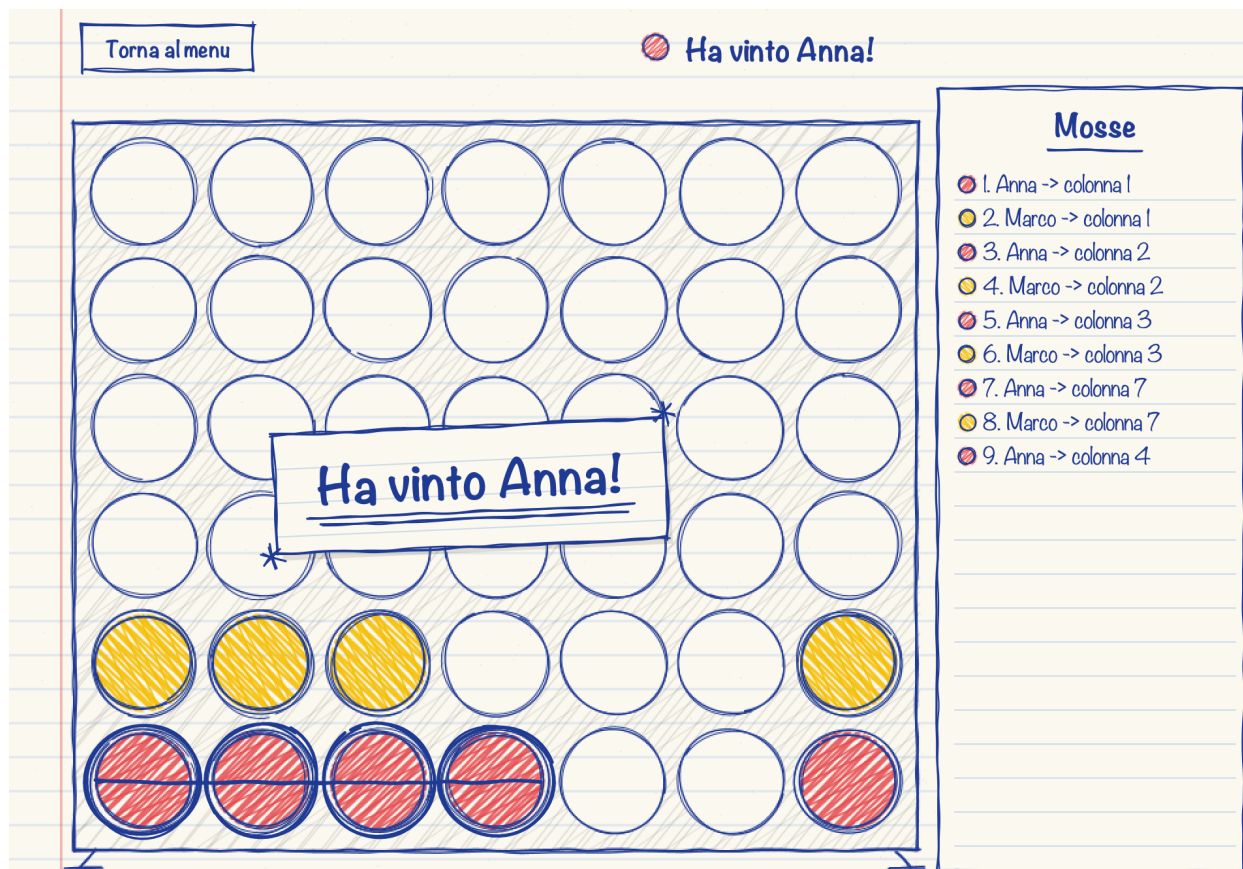


Figura 4: il banner di fine partita, con i quattro gettoni vincenti cerchiati e attraversati da una riga.

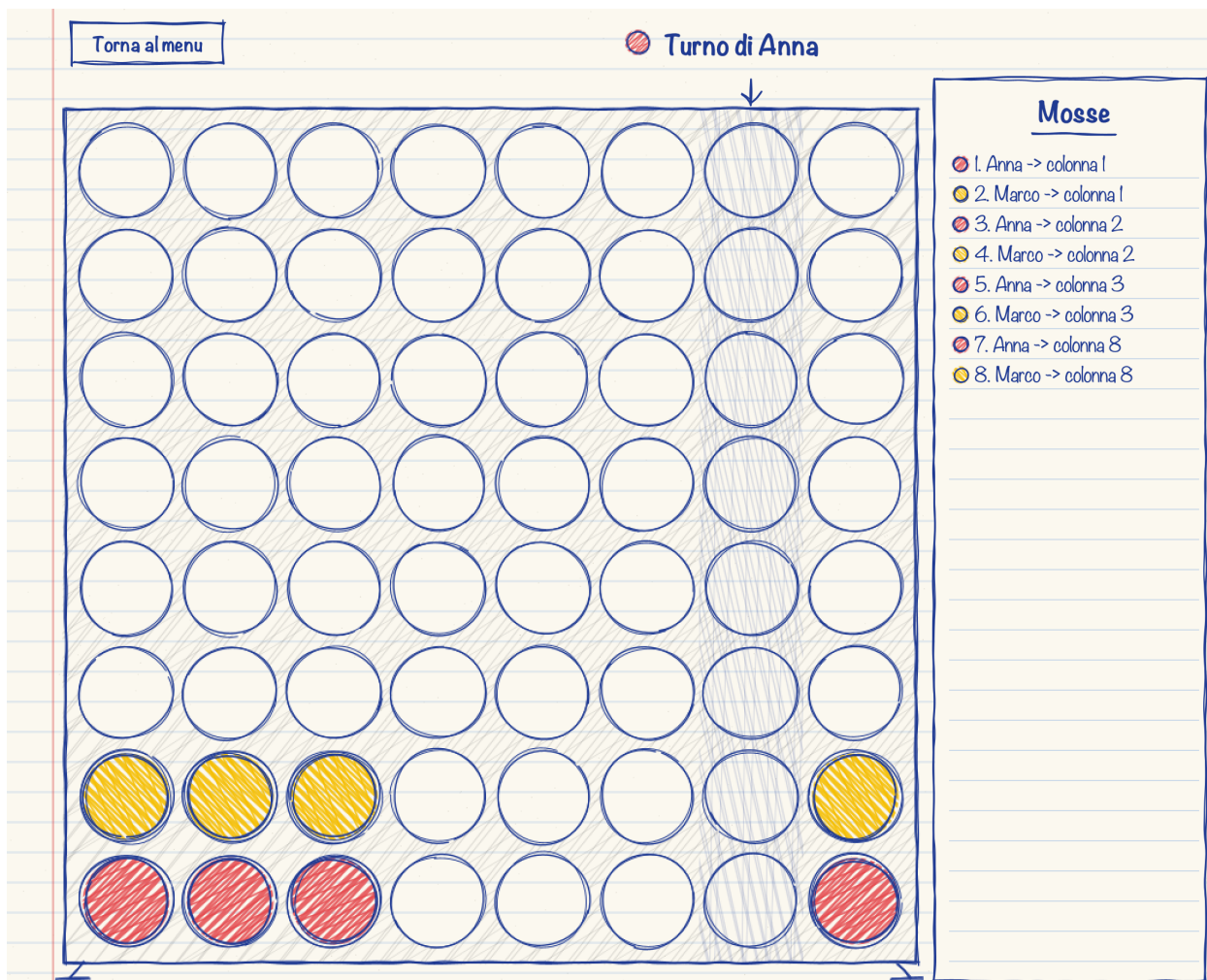


Figura 5: la stessa interfaccia su una griglia 8x8, la dimensione massima consentita.

Descrizione dei problemi incontrati: Un primo problema, puramente grafico, è emerso quando ho notato che i gettoni venivano disegnati fino a toccare, e in alcuni casi superare leggermente, il bordo arrotondato del tabellone blu. La causa era che le dimensioni delle celle venivano calcolate dividendo la larghezza e l'altezza dell'intero pannello per il numero di colonne e di righe, senza tenere conto del margine con cui il tabellone veniva disegnato più piccolo del pannello stesso. Ho risolto il problema calcolando le dimensioni delle celle a partire dallo spazio realmente occupato dal tabellone, margine escluso, e applicando lo stesso margine anche al calcolo della colonna cliccata con il mouse, in modo che il gioco restasse coerente tra disegno e interazione.

Un secondo problema, più delicato, ha riguardato la sincronizzazione tra il thread della partita e quello dell'interfaccia grafica: dovevo far sì che il thread della partita restasse fermo, senza consumare inutilmente il processore, mentre aspettava il clic di un giocatore

umano, ma senza mai bloccare l'interfaccia grafica, che deve restare sempre reattiva. Ho risolto questo problema con il meccanismo wait/notify descritto nei cenni teorici, facendo attenzione a racchiudere entrambi i metodi coinvolti nella parola chiave synchronized per evitare che i due thread si sovrapponevano nell'accesso alla variabile condivisa.

Un terzo problema è emerso durante i test della modalità con l'intelligenza artificiale: su alcune reti, la libreria di rete di Java tentava prima una connessione IPv6 che restava bloccata in silenzio per diversi secondi prima di ripiegare su IPv4, facendo sembrare che il gioco si fosse bloccato mentre in realtà la richiesta sarebbe comunque andata a buon fine. Ho risolto forzando l'uso di IPv4 all'avvio del programma con la proprietà di sistema `java.net.preferIPv4Stack`. In aggiunta, il servizio esterno restituiva talvolta un errore temporaneo di sovraccarico (codice 503): ho quindi aggiunto un meccanismo che ritenta automaticamente la richiesta fino a tre volte prima di considerarla definitivamente fallita.

Un ultimo problema riguardava le animazioni sovrapposte: se una mossa veniva registrata mentre quella precedente stava ancora cadendo (ad esempio nella modalità Umano contro Computer, dove il Computer risponde quasi subito), le due animazioni rischiavano di essere disegnate una sopra l'altra. Ho risolto il problema introducendo una coda che tiene in attesa le mosse successive finché l'animazione corrente non è terminata, facendole partire poi in sequenza.

Rivedendo il progetto a lavoro quasi concluso ho individuato un altro problema, questa volta nell'algoritmo Minimax: la vittoria e la sconfitta avevano sempre lo stesso punteggio, indipendentemente da quante mosse servissero per raggiungerle. Di conseguenza il Computer non distingueva tra una vittoria immediata e una lontana, e quando capiva di aver perso comunque smetteva di difendersi, giocando la prima colonna disponibile. Ho risolto aggiungendo al punteggio di vittoria la profondità rimasta (e sottraendola a quello di sconfitta), così una vittoria vicina vale più di una lontana e una sconfitta lontana vale più di una vicina. Per verificare la correzione ho fatto scegliere la mossa alla vecchia e alla nuova versione su 1223 posizioni casuali: le occasioni di vittoria immediata non sfruttate sono passate da 96 a 0, e le sconfitte immediate che si potevano evitare da 23 a 0.

Altri due problemi riguardavano i tempi dell'interfaccia. Il banner di fine partita compariva mentre l'ultimo gettone stava ancora cadendo, perché l'avviso di fine partita arriva subito dopo l'ultima mossa: ho fatto in modo che banner, cerchiatura e suono di vittoria vengano mostrati solo quando tutte le animazioni in coda sono terminate. Inoltre i suoni arrivavano in ritardo, perché a ogni suono veniva aperta una nuova linea audio, un'operazione che sul mio computer richiedeva circa 86 millisecondi (250 per il primo suono): tenendo invece una sola linea sempre aperta il ritardo misurato è sceso a circa 6 millisecondi.

Infine, tornando al menu durante una partita, il thread della partita precedente restava bloccato per sempre in attesa di un clic: ora viene fermato esplicitamente.

Risultati raggiunti: Al termine del lavoro il programma permette di giocare a Forza 4 su una griglia configurabile da 4 a 8 righe e colonne, in tre modalità diverse: due giocatori umani con nomi personalizzabili, un giocatore umano contro un'intelligenza artificiale locale basata sull'algoritmo Minimax con potatura alpha-beta, oppure un giocatore umano contro un servizio di intelligenza artificiale esterno interrogato via Internet. L'interfaccia mostra un'animazione realistica della caduta dei gettoni, tiene traccia di tutte le mosse giocate in un elenco laterale, riproduce piccoli effetti sonori sintetizzati e, a fine partita, mostra un banner animato con il risultato ed evidenzia la combinazione vincente, il tutto con una grafica in stile disegnato a mano, senza mai interrompere l'esperienza con finestre di dialogo separate. Il programma gestisce correttamente anche i casi limite, come i problemi di rete della modalità AI o le mosse che arrivano durante un'animazione già in corso, senza mai bloccarsi.

Conclusioni: Questo progetto mi ha permesso di applicare concretamente diversi concetti di programmazione a oggetti, in particolare la separazione tra modello e vista e l'uso del polimorfismo attraverso l'interfaccia Giocatore per gestire in modo uniforme avversari molto diversi tra loro. Ho inoltre approfondito la gestione della concorrenza in Java, capendo l'importanza della sincronizzazione tra thread quando un'interfaccia grafica deve restare reattiva mentre attende un evento esterno, e ho implementato da zero un algoritmo di intelligenza artificiale classico come Minimax con potatura alpha-beta, verificandone concretamente l'efficacia giocando contro di esso. Infine, integrando una chiamata a un servizio esterno via HTTP, ho imparato a gestire in modo robusto gli imprevisti tipici della programmazione di rete, come i timeout, gli errori temporanei del servizio e le risposte non previste, senza che questi compromettessero l'esperienza di gioco. Nel complesso, il progetto si è evoluto molto rispetto all'idea iniziale, ed è stata un'occasione per curare non solo la correttezza della logica di gioco ma anche la qualità dell'esperienza utente.